Project 1: SFT LLM for Advanced Reasoning

Jack Wilson Alexander Toskey Jai Ravula Sean Stafford

GitHub Project: https://github.com/JWilson207/SFT-study

INTRODUCTION

Large Language Models, in recent years, have demonstrated strong reasoning capabilities across a multitude of fields and tasks. These capabilities are not only dependent on model architecture, features, and scale, but also on the contents and quality of data the model is trained upon. Supervised Fine Tuning (SFT) is a training method used to adapt a pre-trained model to better perform on specific reasoning tasks. The model is trained on labeled datapoints, those with pairs of input questions and known correct responses. Given resource constraints such as a fixed training budget, curating the data subset selection for training can improve model performance when compared to random sampling.

This project investigates the quantitative impact of curated training-data selection on the reasoning performance of a LLM. We compare two subsets of ~15000 data points each, one randomly selected, and one curated using debloating and scoring techniques. Both subsets are used to fine tune the model using SFT. Model performance is evaluated across eight benchmark datasets: AIME, AMC, CN Math 2024, Kaoyan, MATH, Minerva Math, and OlympiadBench. The results are compared between the original base model, randomly sampled fine-tuned model, and the fine-tuned model using a curated training set.

Environment/ Setup

The base language model used in the experiments was Qwen2.5-3B-Instruct. SFT was done using the LLaMA-Factory, an open-source framework for training, fine-tuning, and well supported for Qwen models. The experiments used **Python version xyz, transformers xyz, llama factory xyz.** Training was performed on **Description of GPU resources/cluster.** These compute environment resources were provided by Virginia Tech Advanced Research Computing. The project provides the training datasets and model checkpoints through Hugging Face.

DATASETS

The experiments use the AceReason dataset in the Supervised Fine Tuning training. Two subsets of ~15,000 examples each compare random sampling to our data selection strategy. Keeping the number of training examples equal lets the experiment investigate the effects of the composition of training data, not the corpus size. The resulting datasets were published to HuggingFace with their associated metadata.

The benchmark datasets provide a diverse range of tasks for the model to be assessed upon. The evaluation data contains 1,944 questions across 8 benchmarks. The benchmark-specific question counts are summarized below:

Benchmark	**Number of questions**	**Generated responses (8 per question)**
AIME	30	240

AMC	40	320
CN Math 2024	30	240
GPQA	198	1,584
Kaoyan	199	1,592
MATH	500	4,000
Minerva Math	272	2,176
OlympiadBench	675	5,400
Total	1,944	15,522

Data Selection Strategy (Advanced15K)

The curated subset was produced by `make_advanced_subset.py`, which applies four stages to the 100,000-example AceReason pool. Each stage targets a known weakness of random sampling for reasoning SFT on a small model.

Stage 1: Format and length limits. Hard constraints remove examples that cannot teach the intended behavior:

- Only responses with a non-empty \boxed{...} answer are kept, matching how the evaluator extracts final answers.
- Trivial traces with fewer than 200 words or fewer than 800 characters are dropped.
- Traces longer than 45,000 characters or 10,000 words (instruction plus response) are dropped. Examples of this length would be truncated at the 16,384-token cutoff during training and lose their final answer.

Stage 2: Domain alignment. Seven of the eight benchmarks are mathematical, so programming problems are removed. They are detected in two ways:

- by metadata: a category or source containing terms such as code, leetcode, taco, codeforces or apps;
- by content: code fences or syntax such as `def f(`, `class Solution`, `#include <` or `std::`.

Stage 3: Reasoning-quality score. Each remaining trace is scored by counting three reasoning behaviors associated with strong long chain-of-thought:

$$score = 2 \times verification + 2 \times backtracking + 1 \times decomposition$$

Behavior	Example phrases
Verification	"let's check", "plugging ... back", verify, substitute, double-check
Backtracking / reflection	"wait", "alternatively", "let me rethink", "however", re-evaluate, contradiction
Decomposition	"Case 1:", "Step 2:", "First, we", "Goal:"

Traces scoring zero are discarded; they would be added back only if the pool fell below 15,000 examples.

Stage 4: Stratified semantic diversity. To keep the quality score from concentrating the subset on a few problem types, selection is spread across topics:

1. Problem statements are embedded with the all-MiniLM-L6-v2 sentence encoder.
2. The embeddings are clustered with KMeans (K = 50, seed 42).
3. The top 300 traces of each cluster are taken, ranked by quality score and then response length.
4. If a cluster has fewer than 300 candidates, the remainder is filled globally from the highest-scoring leftover traces.

The number of examples remaining after each stage is shown below.

Stage	**Examples remaining**
Raw pool	100,000
After Stage 1 (format and length)	62,915
After Stage 2 (no code)	62,866
After Stage 3 (score > 0)	62,843
Selected (Stage 4)	**15,000**

Random baseline. The random subset is drawn uniformly from the same pool by `make_random_subset.py` (15,000 examples, seed 42). Both subsets use the same system prompt, "Please reason step by step, and put your final answer within \boxed{}.", which is mapped to the system field through LLaMA-Factory's `dataset_info.json`.

EXPERIMENTAL SETUP

Both subsets were used to fine-tune Qwen2.5-3B-Instruct with identical settings (configuration files `sft_advanced15k.yaml` and `sft_random15k.yaml`), so that the only difference between the two runs is the training data. The settings are summarized below.

Setting	**Value**
Fine-tuning	Full parameters, DeepSpeed ZeRO-2, bf16
Template / max length	qwen, cutoff length 16,384 tokens
Learning rate	5.0e-6, cosine schedule, no warmup
Effective batch size	8
Epochs	3 scheduled; stopped at step 3,750 (2 epochs)
Attention	SDPA
Seed	42

Checkpointing	Every 250 steps
Hardware	NVIDIA B200 (Virginia Tech ARC, Owl cluster)

The evaluated checkpoint is step 3,750. With 1,875 steps per epoch, this is two-thirds of the scheduled run, where the cosine learning rate is about 25% of its peak; the schedule had therefore not finished decaying.

RESULTS

Both the base model and the fine-tuned version were evaluated across the eight benchmarks. Evaluation included Pass@1 and Pass@8. Performance was evaluated for a single generation, and across multiple responses. The baseline model achieved an average **leaderboard score of 54.3% Pass@8 and 27.5% Pass@1**.

Baseline Vs. Advanced Selection

The table below compares the baseline model against the fine-tuned model on the advanced-selected subset on Pass@8 results. Changes are denoted in percentage points.

Benchmark	**Base model**	**Advanced15K**	**Change**
CN Math 2024	26.7%	40.0%	+13.3 pp
Kaoyan	49.7%	67.8%	+18.1 pp
AMC	77.5%	75.0%	−2.5 pp
Minerva Math	47.8%	42.6%	−5.2 pp
OlympiadBench	48.3%	52.6%	+4.3 pp
GPQA	75.8%	66.7%	−9.1 pp
AIME	13.3%	23.3%	+10.0 pp
MATH	84.0%	86.4%	+2.4 pp
AVG_LB6	**54.3%**	**57.5%**	**+3.2 pp**

The advanced-selected model improved the AVG_LB6 Pass@8 score by 3.2 percentage points. The largest improvements occurred on Kaoyan (+18.1 pp) and CN Math 2024 (+13.3 pp). The results were not fully positive across all benchmarks. The model's Pass@8 score decreased on GPQA (−9.1 pp), Minerva Math (−5.2 pp), and AMC (−2.5 pp).

The table below compares the same baseline model to fine-tuned model performance, but for Pass@1 results.

Benchmark	**Base model**	**Advanced15K**	**Change**
CN Math 2024	11.7%	10.0%	−1.7 pp
Kaoyan	22.9%	27.4%	+4.5 pp
AMC	41.9%	33.8%	−8.1 pp
Minerva Math	30.6%	23.0%	−7.6 pp
OlympiadBench	28.0%	27.4%	−0.6 pp
GPQA	29.9%	17.0%	−12.9 pp
AIME	5.4%	6.2%	+0.8 pp

MATH	65.8%	63.5%	−2.3 pp
AVG_LB6	**27.5%**	**23.1%**	**−4.4 pp**

In contrast to Pass@8 results. The reported Pass@1 score declined by 4.4 percentage points on average.

SUMMARY

This project examined how the choice of training data affects supervised fine-tuning of Qwen2.5-3B-Instruct for mathematical reasoning. From the 100,000-example AceReason pool, we selected 15,000 examples with a four-stage pipeline:

- hard format and length constraints;
- removal of coding problems;
- scoring of reasoning behaviors such as verification, backtracking and decomposition;
- semantic clustering, so the selection covers many problem types.

A random 15,000-example subset was built the same way, and both subsets were trained with identical hyperparameters.

Under the course evaluation protocol, the model fine-tuned on the curated subset raised the six-benchmark average from 54.3% to 57.5% Pass@8 (+3.2 pp). The largest gains were on the hardest competition-style math benchmarks: Kaoyan (+18.1 pp), CN Math 2024 (+13.3 pp) and AIME (+10.0 pp). Performance fell on GPQA (−9.1 pp) and Minerva Math (−5.2 pp). Both rely more on science reasoning, which our math-focused filtering de-emphasized.

Average Pass@1 decreased from 27.5% to 23.1%. The fine-tuned model learned the long, reflective reasoning style of its training data, but it often did not finish within the 32,768-token generation limit. Between 10% and 36% of its responses never reached a final answer.

These results suggest that selecting for reasoning quality helps the model solve harder problems across multiple attempts, but it needs to be balanced with concise answers and broader domain coverage. Future work includes three directions:

- penalizing repetitive self-correction during selection;
- keeping a share of science problems;
- completing the third training epoch and evaluating the random-subset model under the same protocol.